

Real-Time Collaborative Whiteboard

Shared rooms, live drawing events, persistence, permissions, export, and a Kubernetes variant

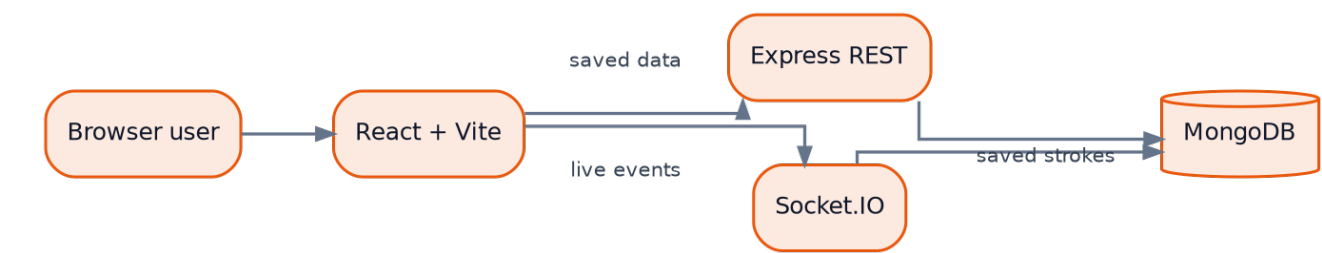
Project context: MARS Open Project, Models and Robotics Section, IIT Roorkee	Status: Implemented full-stack prototype
--	--

A MERN collaboration application where users create or join rooms, draw together through Socket.IO, persist strokes in MongoDB, control public or private access, and export boards as PNG, JPG, or PDF. A second architecture explores service separation with Kubernetes.

Technology stack

React + Vite	Express	Socket.IO	MongoDB
JWT	Canvas tools	Vercel / Render	Kubernetes / Kind

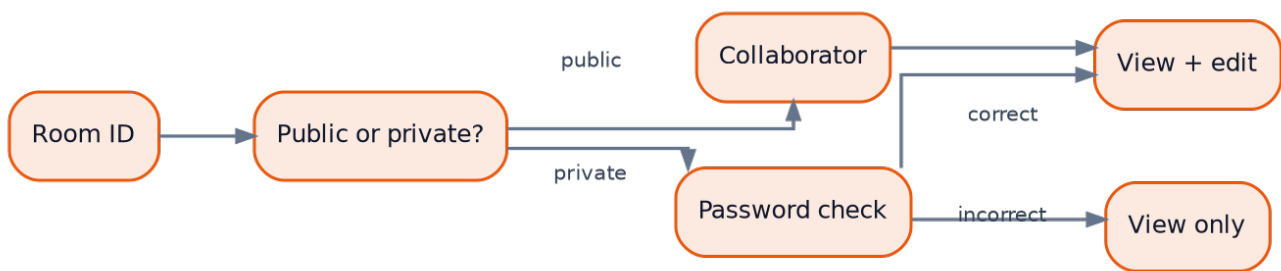
System at a glance



Live collaboration Broadcasts drawing changes to participants in the same room.	Room access Supports shareable room IDs, public rooms, and password-controlled private rooms.
Persistent boards Stores drawing information so a room can be reopened later.	Export and tooling Includes pen, eraser, text, shapes, styling, undo/redo, and downloads.

System design and component ownership

REST endpoints handle durable accounts, rooms, drawings, messages, and user data. Socket.IO carries immediate collaboration events. The standard deployment uses one backend, while the Kubernetes experiment separates responsibilities behind an ingress.



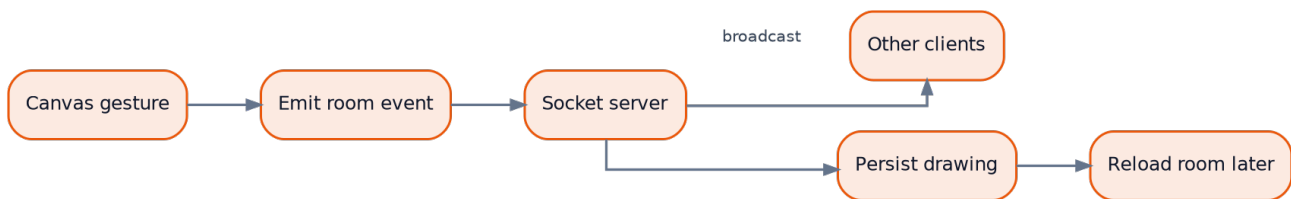
Component	Responsibility
React client	Dashboard, room tools, canvas, users, and room information.
Express REST API	Accounts, room metadata, drawings, messages, and users.
Socket.IO server	Room joins and low-latency drawing updates.
MongoDB	Stores users, rooms, saved strokes, and server-side message data.
Export layer	Produces PNG, JPG, or PDF representations of the board.
Kubernetes variant	Explores ingress and focused backend services in a local Kind cluster.

Key design decisions

- Keep immediate drawing delivery separate from saved REST data.
- Use room IDs as the collaboration boundary.
- Offer a simple monolith for deployment and a separate Kubernetes study for architecture learning.
- Document current security and scaling gaps rather than hiding them.

Core execution flow

A drawing gesture is converted into a room-scoped event, broadcast to other clients, rendered on their canvases, and stored for later retrieval. Access checks decide whether a participant can edit or only view.



Implementation notes

- Public rooms can grant collaboration immediately; private rooms evaluate a password.
- Socket.IO sends live events while MongoDB keeps saved strokes.
- The dashboard supports creating a room or joining by room ID.
- The Kubernetes version separates accounts, rooms, drawings, messages, realtime, and users.
- The first hosted backend request can be slow because of free-tier hosting behavior.

Real-time event pattern

```
client canvas event
-> emit("drawing:update", { roomId, stroke })
-> server joins event to room scope
-> broadcast to other room members
-> clients render the stroke
-> persist saved drawing state for later visits
```

This is a concise representation of the documented Socket.IO and persistence flow.

Engineering review and production path

Engineering strengths

- Useful end-to-end real-time collaboration experience.
- Clear split between immediate events and persistent state.
- Room-level permissions and export functions add practical depth.
- Kubernetes variant demonstrates architectural comparison, not only tooling.

Current limitations

- Several room and user mutations need JWT protection.
- Room passwords are documented as stored and returned in plain text.
- Socket connections and room joins are not JWT-authenticated.
- No implemented automated tests; realtime horizontal scaling needs shared state.

Recommended next steps



Why this is relevant to AI training work

- Requires precise reasoning about event order, shared state, and permission outcomes.
- Provides realistic concurrency and synchronization scenarios for code evaluation.
- Makes it possible to design adversarial tests for duplicate, late, or unauthorized events.
- Shows the ability to explain both a monolith and a service-oriented alternative visually.

Repository:

<https://github.com/SidheshwarSarangal/whiteBoard>

Scope and provenance:

Prepared from the repository README, architecture, realtime-collaboration, Kubernetes, API/data, setup, and current-implementation guides.

